ARTIFICIAL INTELLIGENCE LAB

Python Programs – Aim, Code and Output

10 Practical Programs

Includes: 8-Puzzle, 8-Queen, Water Jug, Crypt-Arithmetic, Missionaries and Cannibals, Vacuum Cleaner, BFS, DFS, TSP and A* Algorithm.

1. 8-Puzzle Problem

Aim: To write a Python program to solve the 8-Puzzle Problem using Breadth-First Search (BFS).

Python Code:

```
from collections import deque

start = (1, 2, 3, 4, 0, 6, 7, 5, 8)
goal = (1, 2, 3, 4, 5, 6, 7, 8, 0)

queue = deque([(start, [start])])
visited = {start}

while queue:
    state, path = queue.popleft()

    if state == goal:
        print("Solution found:")
        for step in path:
            print(step[:3])
            print(step[3:6])
            print(step[6:])
            print()
        break

    zero = state.index(0)
    r, c = divmod(zero, 3)

    for dr, dc in [(0, 1), (0, -1), (1, 0), (-1, 0)]:
        nr, nc = r + dr, c + dc
        if 0 <= nr < 3 and 0 <= nc < 3:
            pos = nr * 3 + nc
            new = list(state)
            new[zero], new[pos] = new[pos], new[zero]
            new = tuple(new)

            if new not in visited:
                visited.add(new)
                queue.append((new, path + [new]))
else:
    print("No solution found")
```

Output:

```
Solution found:
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)

(1, 2, 3)
(4, 5, 6)
(7, 0, 8)

(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

2. 8-Queen Problem

Aim: To write a Python program to solve the 8-Queen Problem using backtracking.

Python Code:

```
def solve(board, row):
    if row == 8:
        for r in board:
            print(" ".join("Q" if c == r else "." for c in range(8)))
        return True

    for col in range(8):
        safe = True
        for r in range(row):
            if (board[r] == col or
                abs(board[r] - col) == abs(r - row)):
```

```
                safe = False
                break

        if safe:
            board[row] = col
            if solve(board, row + 1):
                return True
    return False

board = [-1] * 8
if not solve(board, 0):
    print("No solution")
```

Output:

```
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

3. Water Jug Problem

Aim: To write a Python program to solve the Water Jug Problem using Breadth-First Search (BFS).

Python Code:

```
from collections import deque

A, B, target = 4, 3, 2
queue = deque([(0, 0)])
visited = {(0, 0)}

while queue:
    x, y = queue.popleft()
    print((x, y))

    if x == target or y == target:
        print("Target reached")
        break

    states = [
        (A, y), (x, B), (0, y), (x, 0),
        (x - min(x, B - y), y + min(x, B - y)),
        (x + min(y, A - x), y - min(y, A - x))
    ]

    for state in states:
        if state not in visited:
            visited.add(state)
            queue.append(state)
else:
    print("Target cannot be reached")
```

Output:

```
(0, 0)
(4, 0)
(0, 3)
(4, 3)
(1, 3)
(3, 0)
(1, 0)
(3, 3)
(0, 1)
(4, 2)
Target reached
```

4. Crypt-Arithmetic Problem

Aim: To write a Python program to solve the SEND + MORE = MONEY crypt-arithmetic problem using permutations.

Python Code:

```
from itertools import permutations

letters = "SENDMORY"
for p in permutations("0123456789", len(letters)):
    d = dict(zip(letters, p))

    if d["S"] == "0" or d["M"] == "0":
        continue

    send = int("".join(d[c] for c in "SEND"))
    more = int("".join(d[c] for c in "MORE"))
    money = int("".join(d[c] for c in "MONEY"))

    if send + more == money:
        print("SEND =", send)
        print("MORE =", more)
        print("MONEY =", money)
        break
```

Output:

```
SEND = 9567
MORE = 1085
MONEY = 10652
```

5. Missionaries and Cannibals Problem

Aim: To write a Python program to solve the Missionaries and Cannibals Problem using BFS.

Python Code:

```
from collections import deque

start = (3, 3, 1)
goal = (0, 0, 0)
moves = [(1, 0), (2, 0), (0, 1), (0, 2), (1, 1)]

queue = deque([(start, [start])])
visited = {start}

while queue:
    state, path = queue.popleft()

    if state == goal:
        print("Solution:")
        for step in path:
            print(step)
        break

    m, c, boat = state
    for dm, dc in moves:
        if boat:
            nm, nc, nb = m - dm, c - dc, 0
        else:
            nm, nc, nb = m + dm, c + dc, 1

        if not (0 <= nm <= 3 and 0 <= nc <= 3):
            continue
        if (nm > 0 and nm < nc) or (3 - nm > 0 and 3 - nm < 3 - nc):
            continue

        new = (nm, nc, nb)
        if new not in visited:
            visited.add(new)
            queue.append((new, path + [new]))
else:
    print("No solution")
```

Output:

```
Solution:
(3, 3, 1)
```

```
(3, 1, 0)
(3, 2, 1)
(3, 0, 0)
(3, 1, 1)
(1, 1, 0)
(2, 2, 1)
(0, 2, 0)
(0, 3, 1)
(0, 1, 0)
(0, 2, 1)
(0, 0, 0)
```

6. Vacuum Cleaner Problem

Aim: To write a Python program to implement a simple Vacuum Cleaner Problem using condition-based actions.

Python Code:

```
room = {"A": "Dirty", "B": "Dirty"}
position = "A"

while "Dirty" in room.values():
    print("Vacuum at room", position)

    if room[position] == "Dirty":
        room[position] = "Clean"
        print("Room", position, "cleaned")

    elif position == "A":
        position = "B"
    else:
        position = "A"

print("All rooms are clean")
print(room)
```

Output:

```
Vacuum at room A
Room A cleaned
Vacuum at room A
Vacuum at room B
Room B cleaned
All rooms are clean
{'A': 'Clean', 'B': 'Clean'}
```

7. Breadth-First Search (BFS)

Aim: To write a Python program to implement Breadth-First Search (BFS) on a graph.

Python Code:

```
from collections import deque

graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": [],
    "F": []
}

def bfs(start):
    queue = deque([start])
    visited = {start}

    while queue:
        node = queue.popleft()
        print(node, end=" ")
        for neighbour in graph[node]:
```

```
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

bfs("A")
```

Output:

```
A B C D E F
```

8. Depth-First Search (DFS)

Aim: To write a Python program to implement Depth-First Search (DFS) on a graph.

Python Code:

```
graph = {
    "A": ["B", "C"],
    "B": ["D", "E"],
    "C": ["F"],
    "D": [],
    "E": [],
    "F": []
}

visited = set()

def dfs(node):
    if node not in visited:
        print(node, end=" ")
        visited.add(node)
        for neighbour in graph[node]:
            dfs(neighbour)

dfs("A")
```

Output:

```
A B D E C F
```

9. Travelling Salesman Problem (TSP)

Aim: To write a Python program to solve the Travelling Salesman Problem by checking all possible tours and finding the minimum cost.

Python Code:

```
from itertools import permutations

graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

n = len(graph)
best_cost = float("inf")
best_path = None

for path in permutations(range(1, n)):
    route = (0,) + path + (0,)
    cost = sum(graph[route[i]][route[i + 1]] for i in range(n))

    if cost < best_cost:
        best_cost = cost
        best_path = route

print("Best path:", best_path)
print("Minimum cost:", best_cost)
```

Output:

```
Best path: (0, 1, 3, 2, 0)
```

```
Minimum cost: 80
```

10. A* Algorithm

Aim: To write a Python program to implement the A* search algorithm to find the shortest path in a graph.

Python Code:

```
import heapq

graph = {
    "A": [("B", 1), ("C", 4)],
    "B": [("D", 2), ("E", 5)],
    "C": [("E", 1)],
    "D": [("G", 3)],
    "E": [("G", 2)],
    "G": []
}

h = {"A": 5, "B": 4, "C": 3, "D": 2, "E": 1, "G": 0}

def astar(start, goal):
    queue = [(h[start], 0, start, [start])]
    best_cost = {start: 0}

    while queue:
        f, cost, node, path = heapq.heappop(queue)

        if node == goal:
            print("Path:", path)
            print("Cost:", cost)
            return

        if cost != best_cost.get(node):
            continue

        for neighbour, weight in graph[node]:
            new_cost = cost + weight
            if new_cost < best_cost.get(neighbour, float("inf")):
                best_cost[neighbour] = new_cost
                priority = new_cost + h[neighbour]
                heapq.heappush(queue, (priority, new_cost, neighbour, path + [neighbour]))

    print("No path found")

astar("A", "G")
```

Output:

```
Path: ['A', 'B', 'D', 'G']
Cost: 6
```